Name - Kanishka Patel
Roll no - 26 (D)
Uni. roll no - 2115000505

DSA Assignment - 3 || Module-2

Q1 Give an algorithm for inserting an element into binary tree.

Ans Algorithm:-

1. Start at the root node of the binary tree
2. Compare the value of the element to be inserted with the value of the current Node.
3. If the value of the element is less than the value of the current node, move to the left child of the current Node
4. If the value of the element is greater than the value of the current node move to the right child of the current Node.
5. If the child Node in the desired direction is empty create a new node with the value of the element and attach it to the current node in the corresponding direction.
6. If the child node in the desired direction is not empty repeat steps 2-5 until an empty child Node is reached.
7. Once the element has been inserted, exit the algorithm

Code -

```
Public class Node {
    int value;
    Node left;
    Node right;
    Node(int value) {
        this.value = value;
        left = null;
        right = null;
    }
}
public Node insertNode(Node root, int value) {
    if(root == null) {
        return new Node(value); }
    else if (value < root.value) {
        root.left = insertNode(root.left, value);
    }
    else {
        root.right = insertNode(root.right, value);
    }
    return root;
}
```

Q2 Give an algorithm for finding the size of binary tree

Ans Algorithm :-

1. If the root node is null return 0
2. Recursively calculate the size of the left subtree by calling the function on the left child of the root node
3. Recursively calculate the size of the right subtree by calling the function on the right child of the root node.
4. Add 1 to the sum of the left and right subtree sizes to include the current node in the count.
5. Return the sum calculated in step-4

Code ⇒

```
Public int calculateSize(Node root) {
    if(root == null) {
        return 0; }
    int leftsize = calculateSize(root.left);
    int rightsize = calculateSize(root.right);
    return 1+leftsize+rightsize; }
```

Q3 Give an algorithm for finding the height (or depth) of a binary tree.

Ans Algorithm :-

1. If the root node is null return 0;
2. Recursively calculate the height of the left subtree by calling the function on the left child of the root node.
3. Recursively calculate the height of the right subtree by calling the function on the right child of the root node.
4. Return the maximum of the heights of the left and right subtrees plus 1 to include the current node.

Code:-

```
public int calculateHeight(Node root){
    if(root == null){
        return 0; }
    int leftHeight = calculateHeight(root.left);
    int rightHeight = calculateHeight(root.right);
    return (Math.max(leftHeight, rightHeight) + 1;
    }
```

Ans //4

Write Short Note on binary search tree.

A Binary Search tree (BST) is a type of binary tree in which each node has a value that is greater than or equal to the values of all nodes in its left subtree and less than or equal to the values of all nodes in its right subtree. This property makes it easy to search for a value in the tree, hence the name binary search tree.

Binary Search tree commonly used in computer science because they provide an efficient way to search for and insert elements. The time complexity of these operations is O(log n) on average, where n is the number of elements in the tree. However, in the worst case scenario when the tree is highly unbalanced the time complexity can be O(n).